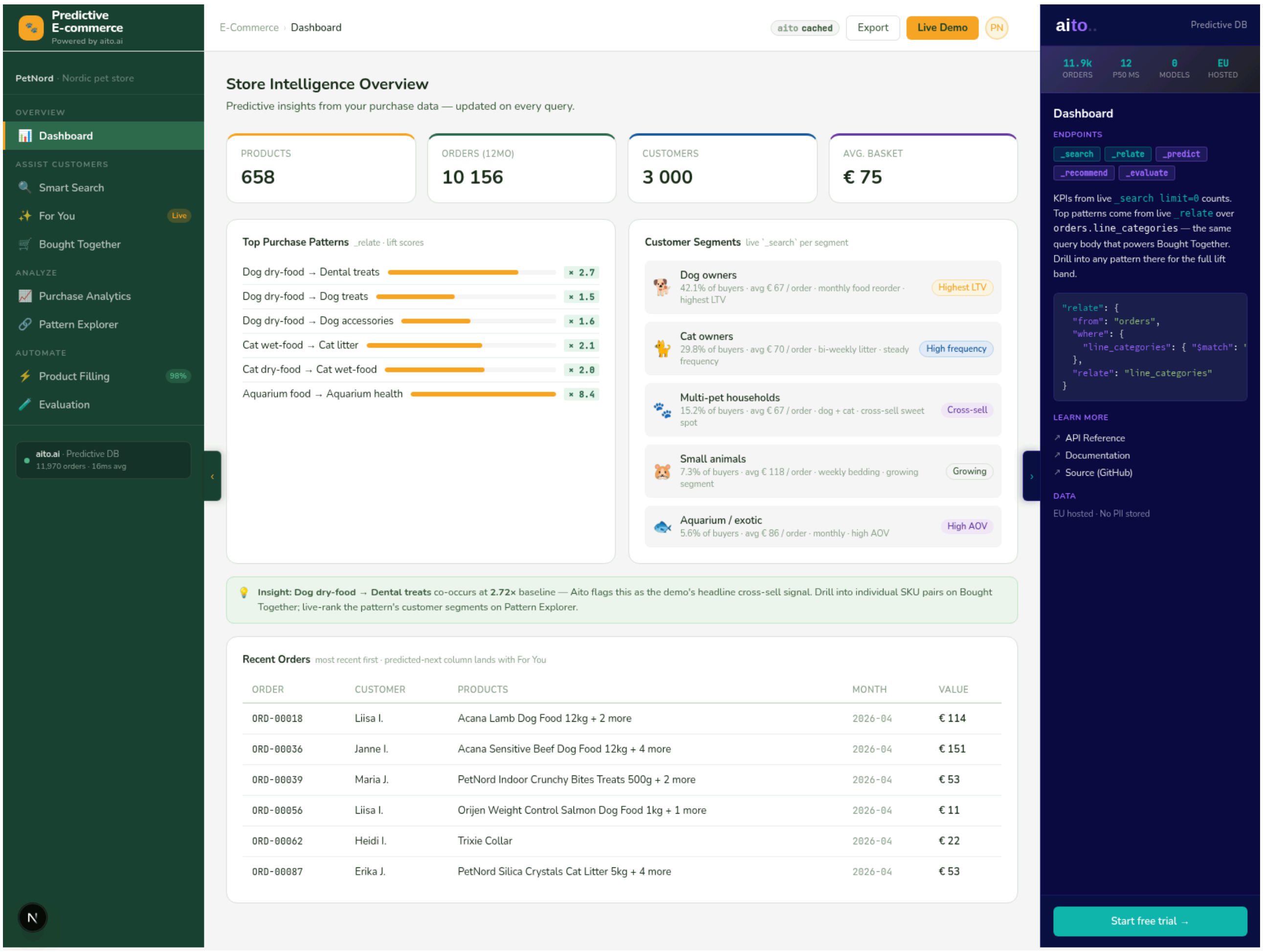
Aito.ai · Predictive Database for E-commerce

The Shop That Learns

From purchase history. Without training. Without rules.

8 production-ready e-commerce features built on a single predictive database. Predictive search · personalised recommendations · co-purchase intelligence · catalog enrichment · evaluation — all from one query API.



The Challenge

E-commerce predictions traditionally mean MLOps. A trained recommendation model that drifts the week after launch. A search re-ranker that needs a GPU pipeline. A return-risk classifier that nobody can explain when it flags a regular customer. Each model starts useful, drifts as the catalog changes, and rots when nobody owns the retraining schedule.

📉 Models drift in days

Recommendations trained on October data feel stale in November when the seasonal catalog rotates. Nobody's monitoring the precision/recall curves.

📦 Catalog data is messy

Half the SKUs have weight, half don't. New supplier feeds arrive with three of the five attributes missing. Manual enrichment costs hours per launch.

❓ Predictions are unauditable

Black-box ranker returned a cat product to a dog owner. Why? The merchandiser has no recourse beyond "the model said so".

The Solution

Aito is a predictive database. Load your purchase history; query for predictions, recommendations, and statistics through SQL-like calls. No model training. No retraining schedule. No MLOps. Every prediction comes with a `$why` decomposition — base rate × pattern lift × pattern lift = final probability — that merchandisers can read and merchandisers can defend.

⚡ Zero training

Upload orders, query immediately. The data is the model; every new transaction improves predictions automatically.

📋 Explainable by design

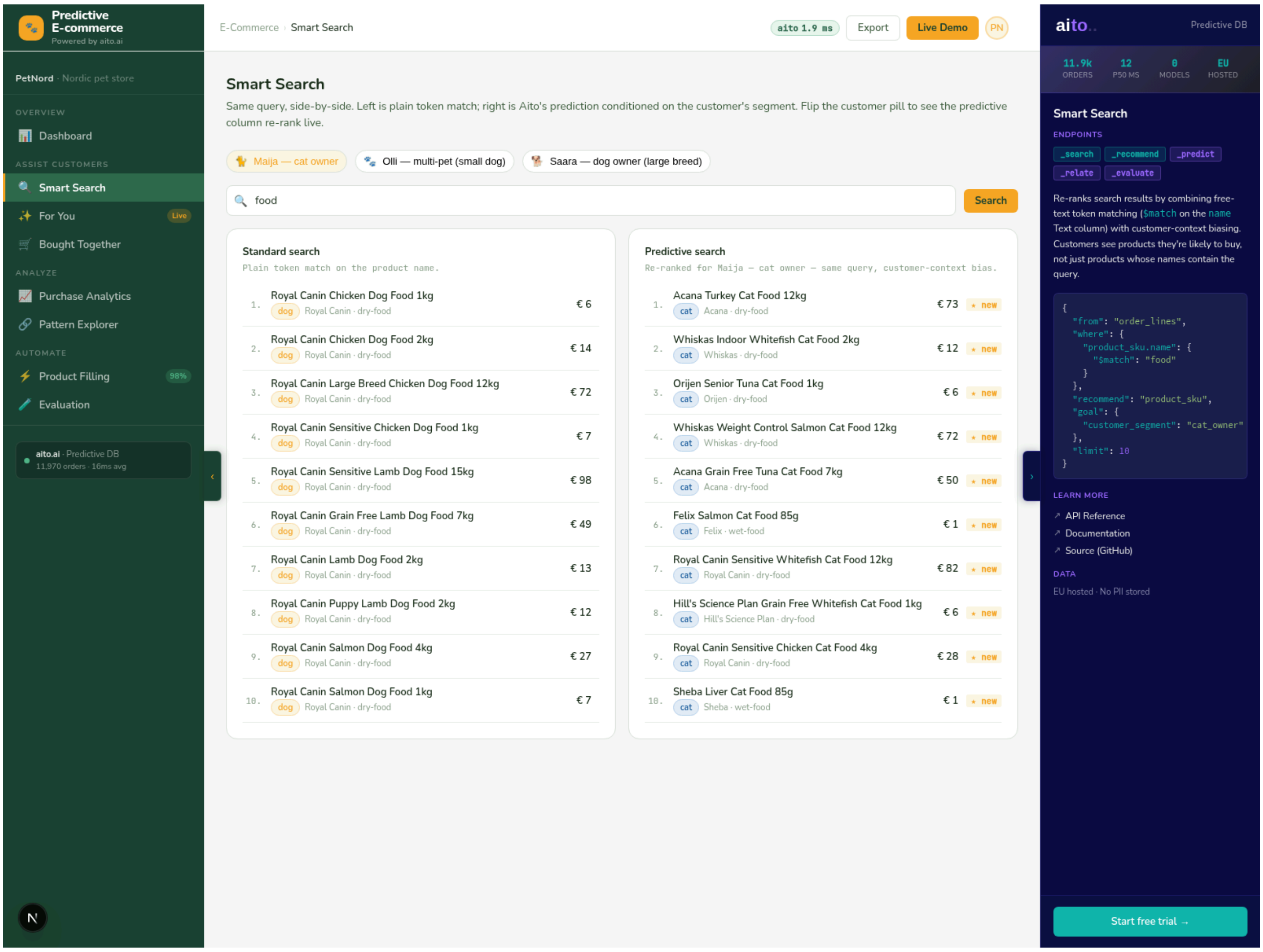
Every `_predict` returns the multiplicative chain that produced it. No black box. Click the ? to see why.

✓ Honest when uncertain

`_evaluate` reports accuracy AND accuracy gain. A 96 % model with +0 pp gain is flagged as honest failure, not shipped as success.

Smart Search — The Rank Flip

The most quotable moment in the demo: type `food` as a generic query. The standard column on the left ranks by token match — dog food at the top because the catalog is dog-heavy. Switch the customer-context pill from Saara (large-breed dog) to Maija (cat owner) and the right column flips entirely. Every cat-food SKU appears with a gold ★ chip — those products weren't in the standard top-10 at all.



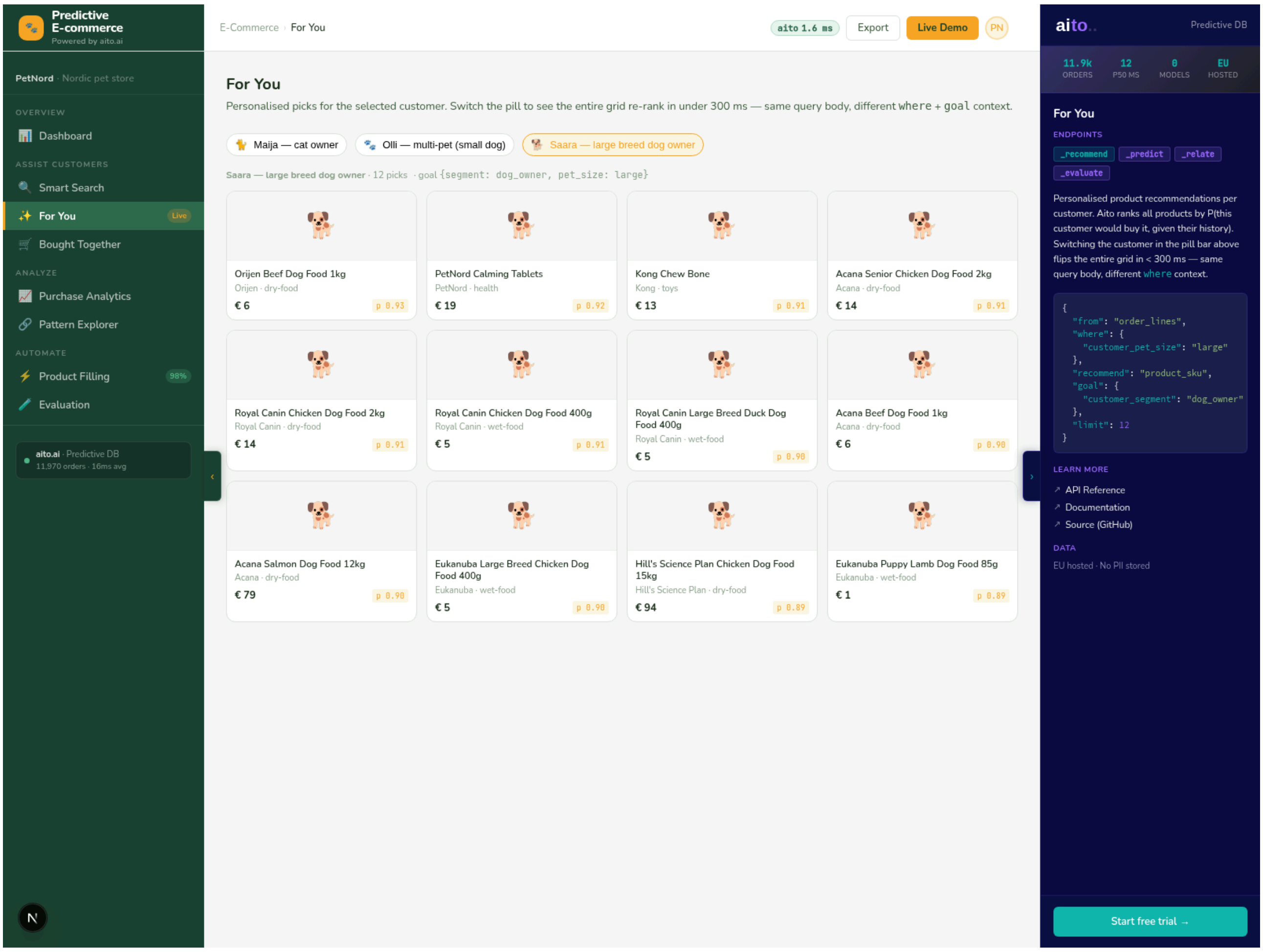
What's happening underneath:

- *Standard*: `_search where {name: {$match: "food"}}` — token relevance only.
- *Predictive*: `_recommend product_sku from order_lines where {product_sku.name: {$match: "food"}} goal {customer_segment: "cat_owner"}` — Aito ranks the same products by P(this segment AND this product) in the live data.
- The `goal` field is the only thing that changes between Maija and Saara. The query body is identical otherwise — and the panel shows it on every switch.

A live demo viewer sees one query, three personas, three completely different result lists — without a single model retrained.

For You — Three Crisply Different Shoppers

No query string, no name filter — just the customer's segment + pet size as the `goal`. The grid re-ranks in under 300 ms per persona flip; the Aito panel updates with the actual `_recommend` body each time.



Three personas in one query shape:

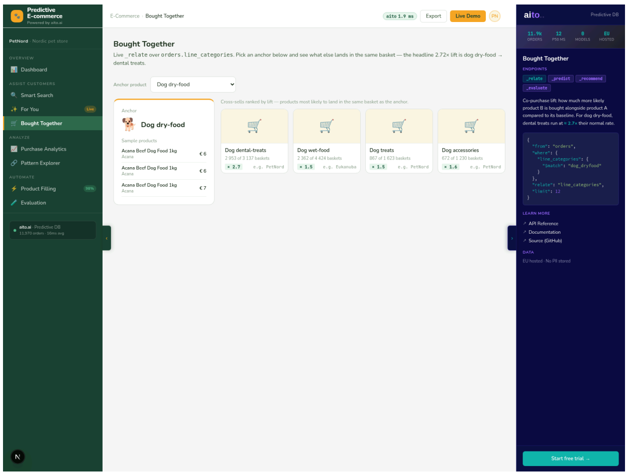
> **Maija** (cat owner) — cat litter, cat dry-food, cat wet-food. Cat consumables top-to-bottom.
> **Olli** (multi-pet, small dog) — dog accessories, grooming, health, toys. Non-food because his segment is the cross-sell shopper.
> **Saara** (large-breed dog) — Acana / Eukanuba / Hill's large-breed kibble. Brand mix swings on `customer_pet_size`.

The split between `where` (filters rows) and `goal` (ranks them) is the load-bearing trick — and Aito's gotcha the demo records: a single multi-field `goal: {segment, pet_size}` collapses onto whichever field dominates. Splitting the constraint gives every persona a sharp result.

Co-Purchase Intelligence

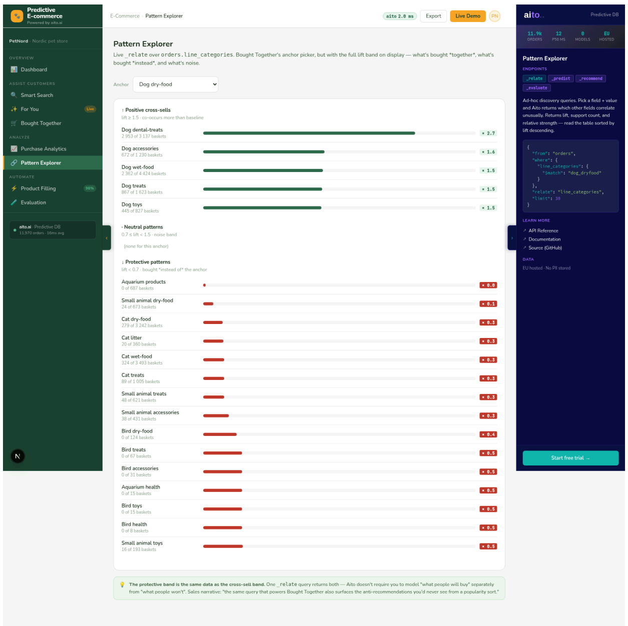
Bought Together

Anchor a product, see four cross-sell tiles with live lift scores. Dog dry-food ⟶ dental treats at **2.72×** baseline. The Aito panel shows the live `_relate` body that drove it.



Pattern Explorer

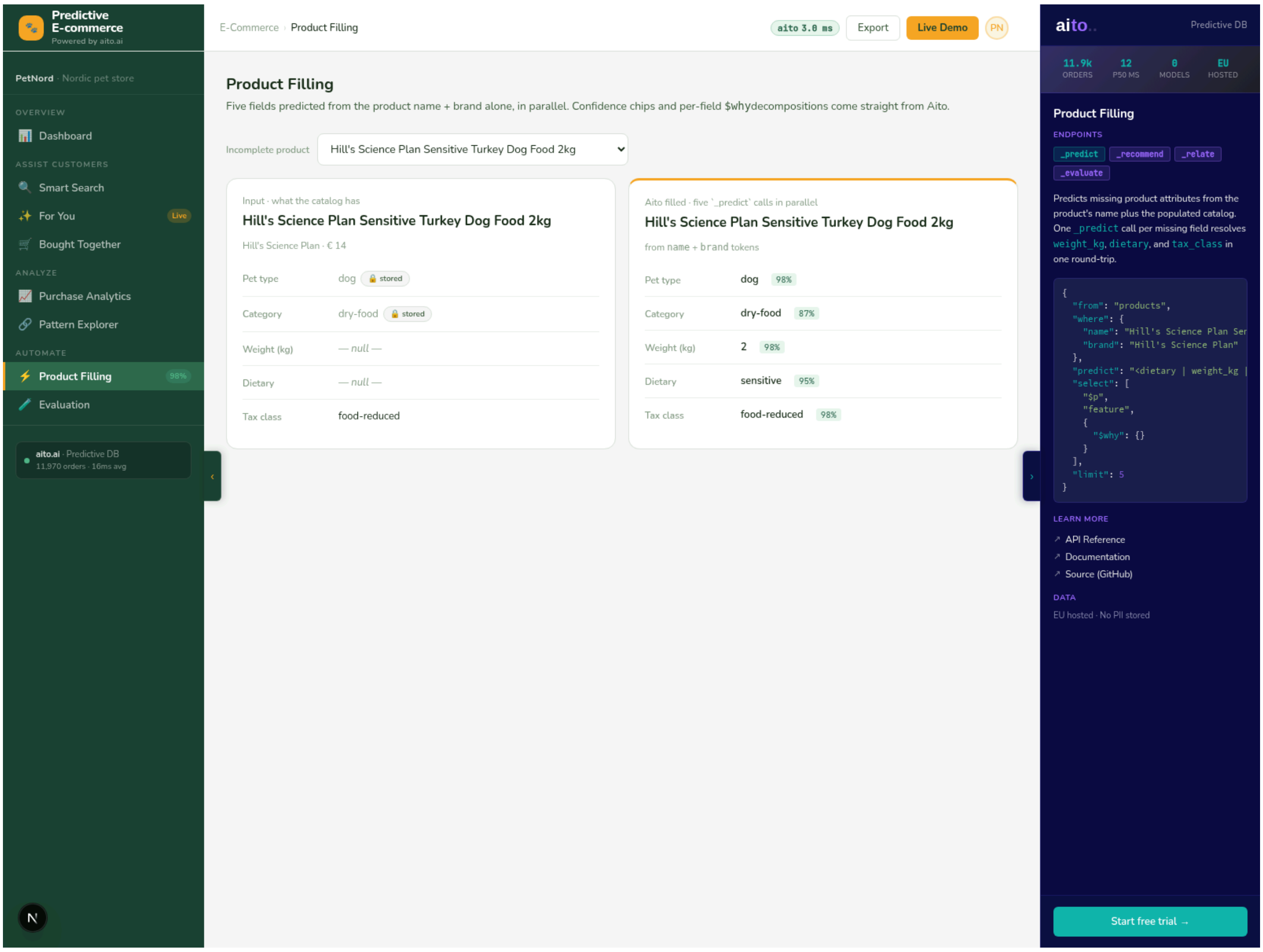
Same `_relate` query, full lift band on display. The positive band is Bought Together's cross-sells. The protective band ("dog dry-food ⟶ cat wet-food × 0.27") is the **anti**-recommendation Aito gives you for free.



The Aito API doesn't expose order-level co-occurrence directly — Aito's `_relate` operates within-row. A denormalised `orders.line_categories` Text column on the schema (each order's `<pet>_<category>` tokens space-separated) makes the query work in one hop. One small column, two views, one of the five demo moments — documented openly in the ADR.

Catalog Enrichment — Five Fields in Parallel

Five `_predict` calls in parallel: pet_type, category, weight_kg, dietary, tax_class. All from `name + brand`. Five round-trips, 500 ms end-to-end. Every prediction returns a confidence chip and a `$why` decomposition.



What this catches:

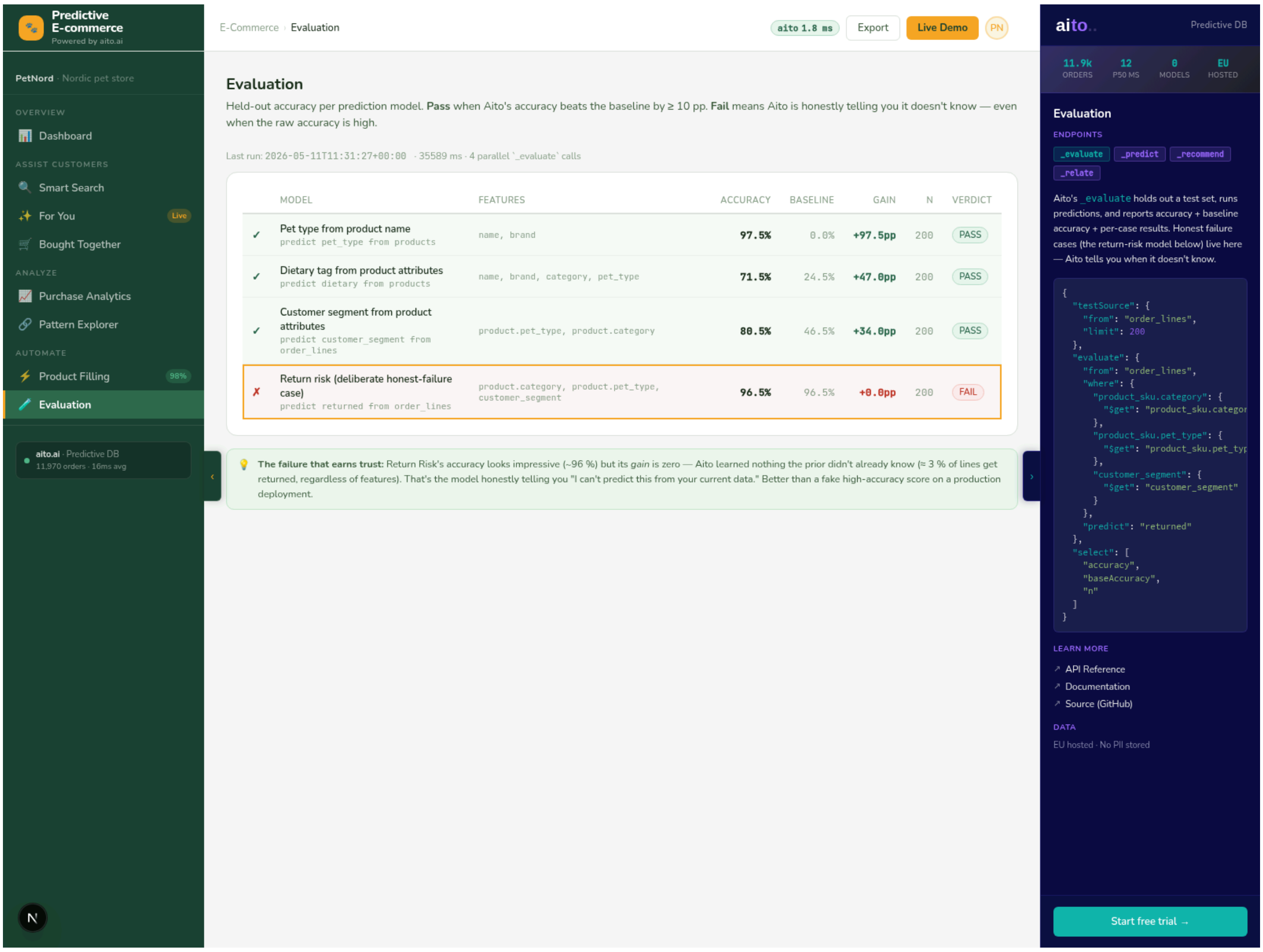
- **Missing weight** — "2kg" in the name tokenises to `weight_kg = 2.0` with 98 % confidence
- **Missing dietary** — "Sensitive" tokenises to `dietary = sensitive` with 95 %
- **Wrong tax class** — `food-reduced` predicted with 98 % from category × brand co-occurrence
- **Two locked fields** (pet_type, category) — already stored; Aito predicts them anyway and the UI tags them 🔒 `stored` for honesty

The `Text` type on `products.name` is load-bearing — Aito tokenises the column and uses individual words ("Sensitive", "2kg", "Dog Food") as features. A schema with `name: String` would collapse the moment to exact-name lookup.

Evaluation — Honest by Design

Four `_evaluate` models. The threshold is `accuracy_gain ≥ +10 pp`. Three pass. One deliberately fails — return-risk gains **+0.0 pp** over the prior. The accuracy column reads 96.5 %; the gain column reads 0.0 pp.

The 96 % isn't earned. About 3 % of order lines get returned regardless of features; predicting "won't be returned" for every line is correct 96.5 % of the time and adds nothing the prior didn't already know. That's what +0 pp gain means — and that's the moment most demos hide.



Why this is the demo's most trust-building moment: every model ships with a held-out evaluation. Aito tells you when its predictions are real signal and when they're a coin flip dressed as accuracy. The return-risk row is the one a CTO would deploy under any naïve metric — and the one Aito refuses to let you ship by labelling it FAIL.

How It Works

📤 1. Connect your data

Catalog + customers + orders + order_lines ⟶ Aito instance. JSON; schema declared in `data_loader.py`. ~53k rows uploads in 50 s.

🔮 2. Query for predictions

Five operators cover all 8 use cases:

- _predict: catalog fill
- _recommend: search + For You
- _relate: co-occurrence
- _search: KPIs + analytics
- _evaluate: honest pass/fail

🔗 3. Integrate

REST API. ~30 ms response time. Sub-1 ms on warm-cache. Two-tier rate limit + persistent cache built in. Drop into your storefront or build standalone.

Architecture at a glance

Backend — Python FastAPI · one service module per view (`overview_service`, `search_service`, `recommend_service`, …)
Frontend — Next.js 16 (App Router) · TypeScript strict · locked Aito side panel on every route
Aito — REST API · `_predict` / `_recommend` / `_relate` / `_search` / `_evaluate`
Cache — Two-layer (in-memory + Aito-backed `prediction_cache`); no-op in `PUBLIC_DEMO=1` mode
Schema — 4 tables; chained-link denormalisation (`order_lines.customer_segment` / `customer_pet_size`, `orders.line_categories`) makes single-hop `_relate` + `_recommend` work cleanly
Public-demo mode — `PUBLIC_DEMO=1` toggles CORS lockdown, two-tier rate limit, memory-only cache, schema 404

Aito gotchas the demo records

The cheatsheet (`docs/aito-cheatsheet.md`) documents seven verified live patterns and seven Aito-API gotchas surfaced while building. Among them: `$match` is required for Text tokenisation (plain `"food"` returns zero hits), multi-field `goal` doesn't behave as AND, Aito tokenises Text on whitespace AND hyphens, `_evaluate` requires both `testSource` AND `evaluate` blocks. Future demo-builders skip the probe rounds.

What This Demo Doesn't Try to Be

This is a **predictive-database reference for e-commerce**, not a complete storefront. The capabilities the demo deliberately omits — and what production would add — are documented openly. A non-exhaustive list:

Checkout / payments — no Stripe wiring, no cart persistence. The demo stops at recommendation; production wires the predictive surfaces into the existing storefront's cart and checkout.

Per-customer history — Aito's `_recommend` underfits on 3 000- customer × few-orders datasets. Segment-level conditioning is the pattern shown; per-individual personalisation needs an order of magnitude more rows and `aito-shopify`'s impressions schema.

Multi-locale catalogs — single Finnish-flavoured catalog. Production adds locale-keyed product tables and routes the `$match` query against the locale's tokeniser.

Image-based search — predictions read off the product name's Text tokens. Image embedding + similarity is a separate capability Aito doesn't ship; production stacks it alongside.

Return-policy enforcement — return-risk is the deliberate honest- failure case. Production wouldn't ship the model with +0 pp gain; it'd surface the evaluation outcome to the merchandising team and return to feature engineering.

Inventory / supply chain — explicitly out of scope; the ERP demo (`aito-erp-demo`) covers that surface. Two demos, two predictive perimeters.

Owning the gaps is more credible than papering over them. Each row above is a real objection raised by e-commerce CTOs reviewing the demo. Production checklist + scaling guidance lives in the ADRs (`docs/adr/`) — 11 architecture decision records documenting every load-bearing choice.

Ready to Try It?

📖 **Read the code**
Apache 2.0 on GitHub. 11 ADRs. ~5k lines Python + TS. Production-quality reference — fork it for your vertical.

🚀 **Run it locally**
./do setup && ./do load-data && ./do dev. Bring your own Aito API key (free tier covers the PetNord 53k rows comfortably).

💬 **Talk to us**
If your e-commerce roadmap has predictive search, recommendations, or catalog enrichment on it, we should talk. EU hosted, no PII stored.

Predictive intelligence for your shop — without the ML pipeline

Aito.ai is a predictive database. Upload orders, query for predictions, ship features. The 8 capabilities in this demo are the same query API you'd use in production — sub-100 ms `_predict` calls, full `$why` explanations, and honest `_evaluate` built in.

hello@aito.ai · aito.ai · github.com/AitoDotAI/aito-ecommerce-demo

Aito.ai builds predictive infrastructure for product teams who want statistical intelligence without standing up an ML team. Open-source demos: aito-demo · aito-accounting-demo · aito-erp-demo · aito-ecommerce-demo.